System Architecture and Integration Design

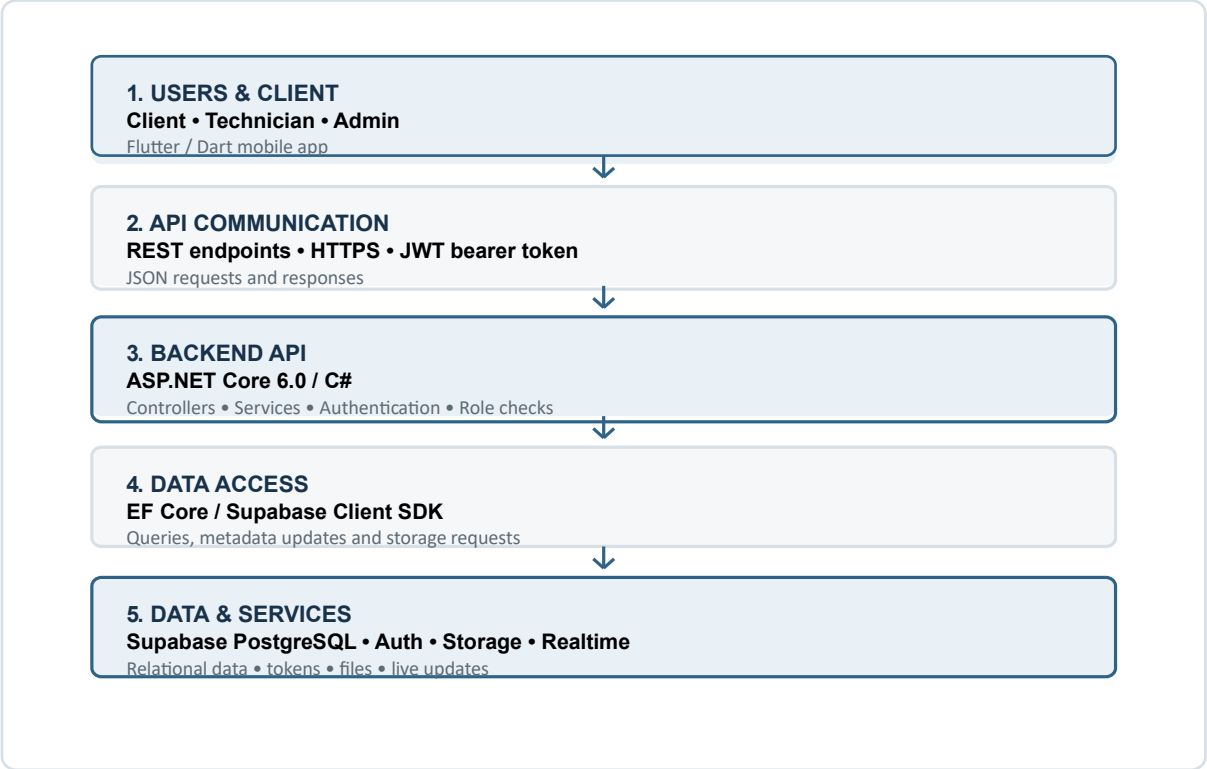
Project: Click Computer and Accessories – Client Support Platform Architecture and Design

1. Purpose and Scope

This section defines the technical architecture of the Client Support Platform. It shows how the Flutter mobile application, the ASP.NET web application and API, and the Supabase backend work together as one integrated solution. It covers the client application, backend API, database, authentication, API communication, security, data flow, file storage, user roles and the Flutter to ASP.NET integration.

The architecture supports only the approved Phase 1 scope: registration and login, device selection, problem or service selection, support request (ticket) submission, request status tracking, communication about a request, technician assignment, and administrative controls.

2. System Architecture Diagram



Communication summary (WHO to WHAT, through WHICH technology, to WHICH data/service):

From (who)	To (what)	Through (technology)	Data / service reached
Client	Flutter mobile app	Touch interface, Dart	Client screens
Flutter app	ASP.NET Core API	REST, JSON, HTTPS,	Tickets, devices,
From (who)	To (what)	Through (technology)	Data / service reached

		JWT bearer token	services, comments, profile
Technician / Admin	ASP.NET web application	Browser, HTTPS, authenticated session	Dashboard, ticket management, assignment, users
ASP.NET web application	ASP.NET business and API layer	Internal service calls	Same business rules as the mobile app
ASP.NET backend	Supabase PostgreSQL	EF Core with Npgsql, TLS	Profiles, Tickets, Assets, Service Catalog, Comments
ASP.NET backend	Supabase Authentication	Supabase client (server side)	Accounts, password verification, JWT issue
ASP.NET backend	Supabase Storage	Supabase client, signed URLs	Ticket attachments (private bucket)
Supabase Realtime (where applicable)	Flutter app	Secure WebSocket, read-only	Ticket status change notifications

3. Component Explanation

3.1 Client application (Flutter / Dart)

The Flutter app is the client-facing interface for customers of Click Computer and Accessories. It lets a client register, log in, select a device, select a problem or service, submit a support request, view request status and communicate about a request. It is designed for simple navigation and minimal steps. The app holds no database credentials. It stores only the user's JWT in secure device storage and sends it to the API with each request.

3.2 Web application and backend API (ASP.NET Core 6.0 / C#)

The ASP.NET Core solution has two parts that share the same business logic:

- **REST API** used by the Flutter app. It validates input, checks the JWT and role, applies business rules and writes to the database.
- **Web application (admin and technician interface)** used by Admins and Technicians for the dashboard, ticket management, technician assignment, technician workflow and user management.

The layers are: Controllers (API endpoints and web pages), Services (business rules such as ticket status transitions and assignment rules), Data access (EF Core DbContext and the Supabase client) and Models/DTOs (data transfer objects, so internal entities are never exposed directly).

3.3 Database (Supabase PostgreSQL)

Supabase PostgreSQL is the single system of record. Core structures are Profiles (linked to the authentication user and holding the role), Tickets, Assets (client devices), Service Catalog (device categories and problem/service types) and Comments.

3.4 Authentication (Supabase Authentication with JWT)

Supabase Authentication stores credentials and issues JSON Web Tokens. Passwords are hashed and managed by Supabase and are never stored or handled by our application tables. Each user's role (Client, Technician or Admin) is held in the Profiles table and is included in the authorisation checks of the API.

3.5 File storage (Supabase Storage)

Optional ticket attachments such as photos of a faulty device are stored in a private Supabase Storage bucket. Files are uploaded through the ASP.NET API, which checks file type and size and then saves the file path against the ticket. Files are viewed only through short-lived signed URLs issued after the API confirms the user may see that ticket.

4. User Roles

Role	Interface	Purpose
Client	Flutter mobile app	Register, log in, submit tickets, view own ticket status, communicate on own tickets
Technician	ASP.NET web application	View and manage tickets assigned to them, update status, communicate on assigned tickets
Admin (including support operator duties)	ASP.NET web application	View all tickets, manage users and operations, administrative controls

The permissions per function are defined in the Access Control Matrix by Ndlovu N. The architecture enforces them in the API and web layer and, as a second layer, in Supabase Row Level Security.

5. API Communication and Flutter to ASP.NET Integration

- **Protocol:** REST over HTTPS (TLS). Plain HTTP is not accepted.
- **Format:** JSON request and response bodies.
- **Authentication:** `Authorization: Bearer <JWT>` header on every protected request.
- **Login path:** Flutter sends credentials to the API login endpoint. The API verifies them with Supabase Authentication and returns the JWT. Flutter stores it securely and attaches it to later calls.
- **Validation:** The API validates every request body, rejects invalid data with clear messages, and checks the user's role before running the action.

- **Responses:** Standard HTTP status codes (200/201 success, 400 validation error, 401 not authenticated, 403 not permitted, 404 not found, 500 server error) with a short user-readable message, which the Flutter app shows as clear confirmation or error feedback.

6. Data Flow Explanation

6.1 Login

1. The client enters email and password in the Flutter app.
2. Flutter sends them to the API over HTTPS.
3. The API asks Supabase Authentication to verify the credentials.
4. On success Supabase returns a JWT. The API reads the user's role from Profiles and returns the token and role.
5. Flutter stores the token securely and shows the client home screen. On failure a clear error message is shown.

6.2 Submit a support ticket

1. The client selects a device and a problem or service (loaded from the Service Catalog through the API).
2. The client adds a description and optional photo and taps Submit.
3. Flutter sends the request with the JWT to POST /api/tickets.
4. The API validates the JWT, confirms the Client role, validates the data, and sets the owner from the token (the client cannot submit on behalf of someone else).
5. EF Core inserts the Ticket record with status "Open".
6. The assignment service automatically selects an available Technician, saves the assignment on the ticket and sets the status to “Assigned”.
7. If a photo is included, the API stores it in Supabase Storage and saves the file path.
8. The API returns the ticket reference, and Flutter shows a confirmation.

6.3 View ticket status

Flutter calls GET /api/tickets/mine or GET /api/tickets/{id}. The API returns only tickets that belong to the authenticated client, and Flutter displays the current status.

6.4 Technician handles an assigned ticket

The Technician sees only tickets assigned to them. The API checks that the ticket is assigned to that technician before allowing a status update or comment.

6.5 Client communicates about a ticket

The client or staff member posts a comment through the API. The API confirms the user is the ticket owner, the assigned technician or an Admin, then saves the comment and links it to the ticket. The other party sees it on their next load or through Realtime.

7. Security Design

Area	Design decision

Controlled database access	Flutter never connects to the database directly and has no service key. All writes go through the ASP.NET API, which applies validation and role checks.
Transport security	HTTPS/TLS for Flutter to API, browser to web app and backend to Supabase.
Authentication	Supabase Authentication issues JWTs. Passwords are hashed by Supabase and never stored by the application.
Authorisation	Every endpoint and page requires a valid JWT and the correct role. Ownership is checked on every ticket request (client sees own tickets, technician sees assigned tickets, Admin sees all).
Secret management	The Supabase service role key, database connection string and JWT secret are held only in server-side configuration or environment variables, never in the Flutter app or the GitHub repository.
Row Level Security	RLS is enabled on all tables as a second layer so that direct access with a public key cannot read or change other users' data. Policy details are in the Supabase and Access Control section (Molefe B and Ndlovu N).
Input validation	Server-side validation of all inputs. EF Core parameterised queries prevent SQL injection. DTOs prevent over-posting.
File access	Private bucket, allowed file types and size limit, signed URLs with short expiry, access only after the ticket permission check.
Data protection	Only the minimum personal information is stored. Clients cannot see other clients' data. Technicians see only the client details needed for the assigned ticket.
Error handling	Users see short, clear messages. Technical details and stack traces are logged on the server only.

Area	Design decision
Data consistency	Foreign keys and constraints in PostgreSQL, plus transactions in EF Core when a change touches more than one table (for example assignment plus status change).
Session handling	JWTs have a limited lifetime. Expired tokens return 401 and the user is asked to log in again.

8. Technology Justification

Technology	Why it was chosen
Flutter / Dart	One codebase for Android and iOS. Fast development of simple, consistent screens suited to clients with limited digital experience. Good HTTP and secure-storage support.
ASP.NET Core 6.0 / C#	Mature framework for both a REST API and a server-rendered web interface in one solution. Built-in JWT validation, role-based authorisation and dependency injection. Strong typing reduces errors. Part of the module's required technology stack.
Supabase PostgreSQL	Reliable relational database that suits the strongly related data (clients, tickets, technicians, devices). Hosted, so no server administration.
Supabase Authentication	Secure, ready-made account management and JWT issuing, avoiding a custom and riskier password system.
Supabase Storage	Built-in private file storage with signed URLs, integrated with the same project and access policies.
REST + JSON + HTTPS	Simple, widely supported and easy to test. Keeps Flutter and ASP.NET loosely coupled so each can change independently.

Why a backend API sits between Flutter and Supabase: it enforces business rules in one place, protects secrets, stops unrestricted database writes from the mobile app, and lets the web and mobile applications share exactly the same rules.

9. Requirements Traceability (Architecture View)

Phase 1 requirement	Architecture element that supports it
Registration and login	Flutter login screen, auth endpoints, Supabase Authentication, Profiles
Device selection	Catalog endpoint, Service Catalog and Assets tables
Problem / service selection	Catalog endpoint, Service Catalog table
Support request submission	POST /api/tickets, Tickets table, optional Storage
Status tracking	GET ticket endpoints, Tickets table, optional Realtime
Communication about a request	Comment endpoint, Comments table
Technician assignment	Assignment endpoint, ASP.NET admin dashboard, Tickets and Profiles
Administrative controls	ASP.NET web application, role-based authorisation, user management endpoints
Security and role separation	JWT, role checks, RLS, HTTPS, secret management